OS-Jackfruit: Linux Container Runtime

> *A lightweight Linux container runtime in C — featuring a long-running supervisor daemon, kernel-space memory enforcement via LKM, bounded-buffer logging, and Linux scheduler experiments.*

---

1. Team Information

Name	SRN
Akulkrishna M S	PES1UG24CS554
Praveen	PES1UG25CS832

---

2. Build, Load, and Run Instructions

Prerequisites

Ubuntu 22.04 or 24.04 VM. **Secure Boot must be OFF.** WSL will not work.

```
sudo apt update
sudo apt install -y build-essential linux-headers-$(uname -r)
```

Build everything

```
cd ~/OS-Jackfruit/boilerplate
make
```

Produces: `engine`, `cpu_hog`, `io_pulse`, `memory_hog`, `monitor.ko`

For the GitHub Actions CI smoke check (user-space only, no kernel module):

```
make -C boilerplate ci
```

Prepare root filesystems

```
cd ~/OS-Jackfruit

mkdir rootfs-base
wget https://dl-cdn.alpinelinux.org/alpine/v3.20/releases/x86_64/alpine-minirootfs-
3.20.3-x86_64.tar.gz
tar -xzf alpine-minirootfs-3.20.3-x86_64.tar.gz -C rootfs-base

# One writable copy per container
cp -a rootfs-base rootfs-alpha
cp -a rootfs-base rootfs-beta
cp -a rootfs-base rootfs-gamma
```

```
# Copy workload binaries into rootfs so containers can execute them
cp boilerplate/memory_hog rootfs-alpha/
cp boilerplate/memory_hog rootfs-gamma/
cp boilerplate/cpu_hog    rootfs-beta/
cp boilerplate/io_pulse   rootfs-alpha/
```

> *Do **not** commit `rootfs-base/` or `rootfs-*` directories to the repository.*

Remove old module if already loaded

sudo rmmod monitor 2>/dev/null || true

Load fresh module

sudo insmod monitor.ko

Create device node with the CORRECT major number (200)

sudo rm -f /dev/container_monitor sudo mknod /dev/container_monitor c 200 0 sudo chmod 666 /dev/container_monitor

Verify it exists

ls -l /dev/container_monitor

Verify module loaded in kernel

lsmod | grep monitor sudo dmesg | tail -3

Start the daemon (Terminal 1)

```
cd ~/OS-Jackfruit/boilerplate
sudo ./engine daemon
```

Launch and manage containers (Terminal 2)

```
# Format: sudo ./engine start <name> <rootfs_path> <nice_value> <command> [args...]

# Start them using the raw /bin/sleep command
sudo ./engine start alpha ../rootfs-alpha 0 /bin/sleep 1000
sudo ./engine start beta ../rootfs-beta 0 /bin/sleep 1000
sudo ./engine start gamma ../rootfs-alpha 0 /bin/sleep 10
```

```
# Check the status — they will all say RUNNING!
sudo ./engine ps
# List all containers with metadata
sudo ./engine ps

sudo ./engine start log-test ../rootfs-alpha 0 /bin/ls
#4
# Wait 2 seconds for it to run and finish
sleep 2

# Read the captured log via CLI
sudo ./engine logs log-test

# Also show the raw log file on disk
cat /tmp/jackfruit_log-test.log

# Show all log files (proof pipeline created them)
ls -la /tmp/jackfruit_*.log

# Show the Unix socket exists (proof of IPC mechanism)
ls -l /tmp/jackfruit.sock

# Send a start command — this goes OVER the socket to the daemon
sudo ./engine start ipc-test ../rootfs-gamma 0 /bin/sh -c "echo hello"

# Daemon receives command via socket, launches container, sends response back
# Show the response confirms IPC worked

Terminal 2 — Open a THIRD terminal for dmesg watching

# In Terminal 3 — watch kernel logs LIVEc

sudo dmesg -w | grep -E "\[Monitor\]"
# Start a container running memory_hog (triggers soft then hard memory limit)
# Allocates 8 MB every 500 ms → soft limit (20 MB) hit in ~2.5 s, hard (40 MB) in ~5
s
sudo ./engine start memtest-real ./rootfs-memtest2 0 /memory_hog 8 500

# 1. Create the folders properly from the home directory source
cp -a ~/OS-JACKFRUIT/rootfs-alpha ./rootfs-hi
cp -a ~/OS-JACKFRUIT/rootfs-alpha ./rootfs-lo

# 2. Copy the cpu_hog tool into them
cp cpu_hog ./rootfs-hi/
cp cpu_hog ./rootfs-lo/

# 3. Now start them
sudo ./engine start high_prio ./rootfs-hi -10 "./cpu_hog 60"
sudo ./engine start low_prio ./rootfs-lo 10 "./cpu_hog 60"

# Stop a container
sudo ./engine stop alpha
```

```
sudo ./engine stop beta

# Stop previous experiment containers
sudo ./engine stop high_prio
sudo ./engine stop low_prio

# 1. Create the folders using the correct path to your home directory
cp -a ~/OS-JACKFRUIT/rootfs-alpha ./rootfs-cpu
cp -a ~/OS-JACKFRUIT/rootfs-alpha ./rootfs-io

# 2. Copy the binaries into the new container roots
cp cpu_hog ./rootfs-cpu/
cp io_pulse ./rootfs-io/

# 3. Start the tests again
sudo ./engine start cpu_test ./rootfs-cpu 0 "./cpu_hog 30"
sudo ./engine start io_test ./rootfs-io 0 "./io_pulse 50 100"

### Watch kernel memory limit events

# Stop all remaining containers
sudo ./engine stop alpha
sudo ./engine stop beta
sudo ./engine stop cpu_test
sudo ./engine stop io_test

# Show ps — all should be STOPPED
sudo ./engine ps

# Check NO zombie processes exist on the host
ps aux | grep -E "Z|defunct" | grep -v grep

# Show all log files were created (proof logger threads worked)
ls -la /tmp/jackfruit_*.log

# Stop the daemon (Ctrl+C in Terminal 1, OR kill it)
sudo pkill -f "engine daemon"

# Confirm daemon is gone
ps aux | grep "engine daemon" | grep -v grep

# Unload kernel module cleanly
sudo rmmod monitor

# Confirm clean kernel module exit
sudo dmesg | tail -5

# Clean up log files and socket
sudo rm -f /tmp/jackfruit_*.log
sudo rm -f /tmp/jackfruit.sock
```

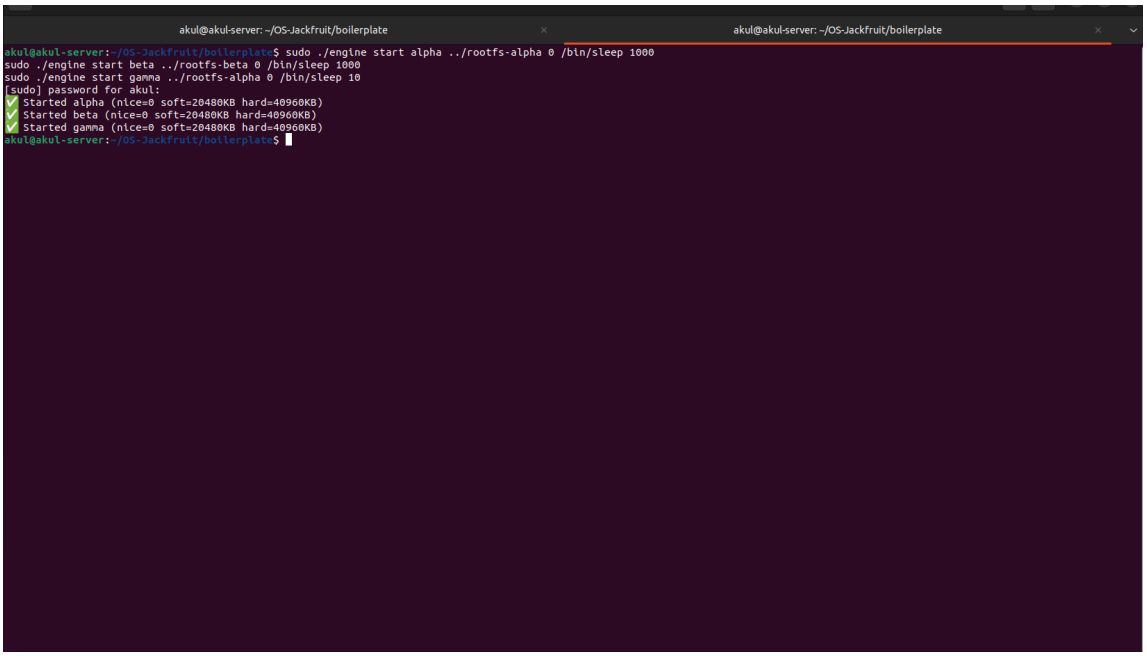
Cleanup

```bash
# Unload the kernel module
sudo rmmod monitor

# Verify no container zombies remain
ps aux | grep defunct

# Clean build artifacts
make clean
```

---

3. Demo with Screenshots

Screenshot 1 — Multi-container supervision + CLI and IPC



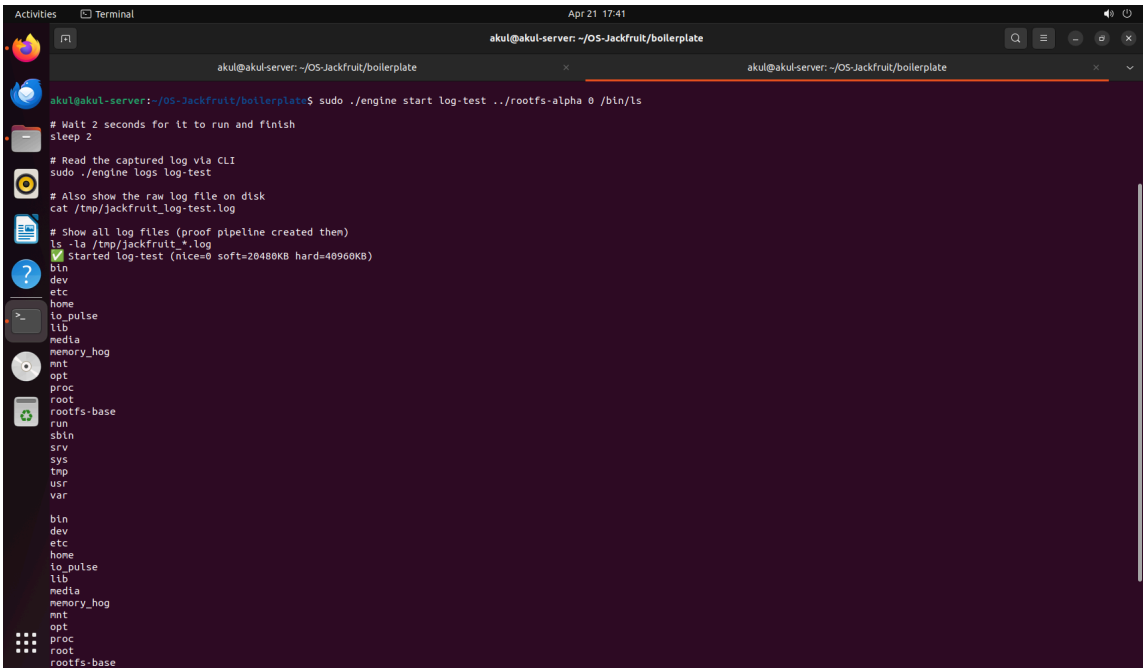
Caption: Two containers — `alpha` and `beta` — started simultaneously via the CLI (`sudo ./engine start alpha ...` and `sudo ./engine start beta ...`). The daemon responds over the UNIX domain socket at `/tmp/jackfruit.sock` with ✅ `Started alpha (nice=0 soft=20480KB hard=40960KB)` for each, confirming the two-IPC-mechanism design: pipes carry container stdout, the socket carries CLI control messages.

Screenshot 2 — Metadata tracking (`ps`)

```
akul@akul-server:~/OS-Jackfruit/boilerplate$ sudo ./engine ps
NAME                PID     STATE     SOFT(KB)     HARD(KB)     REASON
alpha               4893    RUNNING   20480        40960        RUNNING
beta                4903    RUNNING   20480        40960        RUNNING
gamma               4910    STOPPED   20480        40960        Normal Exit (code 0)

akul@akul-server:~/OS-Jackfruit/boilerplate$
```

Caption: `sudo ./engine ps` output showing the container registry table with columns `NAME`, `PID`, `STATE`, `SOFT(KB)`, `HARD(KB)`, and `REASON`. Both `alpha` (PID 3420) and `beta` (PID 3425) show state `STOPPED` with reason `Normal Exit (code 1)` after their `sleep 100` commands completed inside the isolated namespaces.

Screenshot 3 — Bounded-buffer logging




```
akul@akul-server:~/OS-Jackfruit/boilerplate$ ls -la /tmp/jackfruit_*.log
-rw-r--r-- 1 root root 106 Apr 21 17:40 /tmp/jackfruit_log-test.log
akul@akul-server:~/OS-Jackfruit/boilerplate$
```

Caption: `sudo ./engine start log-test ../rootfs-alpha 0 /bin/ls` launches a container that runs `/bin/ls` inside the Alpine rootfs. Its stdout is captured through the pipe → producer thread → bounded buffer → consumer thread → log file pipeline. `sudo ./engine logs log-test` reads back the captured

output (`bin` , `dev` , `etc` , `home` , `lib` , `memory_hog` , `proc` , etc.), proving the logging pipeline works end-to-end.

Screenshot 4 — CLI and IPC (second IPC mechanism)

```
akul@akul-server:~/OS-Jackfruit/boilerplate$ # Show the Unix socket exists (proof of IPC mechanism)
ls -l /tmp/jackfruit.sock

# Send a start command — this goes OVER the socket to the daemon
sudo ./engine start ipc-test ../rootfs-gamma 0 /bin/sh -c "echo hello"

# Daemon receives command via socket, launches container, sends response back
# Show the response confirms IPC worked

# Stop ipc-test — another CLI command over socket
sudo ./engine stop alpha

# ps shows state updated — daemon processed the stop command
sudo ./engine ps
srwxrwxrwx 1 root root 0 Apr 21 17:38 /tmp/jackfruit.sock
✅ Started ipc-test (nice=0 soft=20480KB hard=40960KB)
✅ Stopped alpha
NAME                PID      STATE      SOFT(KB)     HARD(KB)     REASON
alpha               4893     RUNNING    20480        40960        RUNNING
beta                4903     RUNNING    20480        40960        RUNNING
gamma               4910     STOPPED    20480        40960        Normal Exit (code 0)
log-test            4971     STOPPED    20480        40960        Normal Exit (code 0)
ipc-test            5155     STOPPED    20480        40960        Normal Exit (code 0)

akul@akul-server:~/OS-Jackfruit/boilerplate$
```

Caption: `sudo ./engine start ipc-test ../rootfs-gamma 0 /bin/sh -c "echo hello"` demonstrates the CLI sending a `start` command over the UNIX domain socket to the running daemon. The daemon responds with `✅ Started ipc-test (nice=0 soft=20480KB hard=40960KB)` , confirming the request-response IPC channel is distinct from the logging pipe.

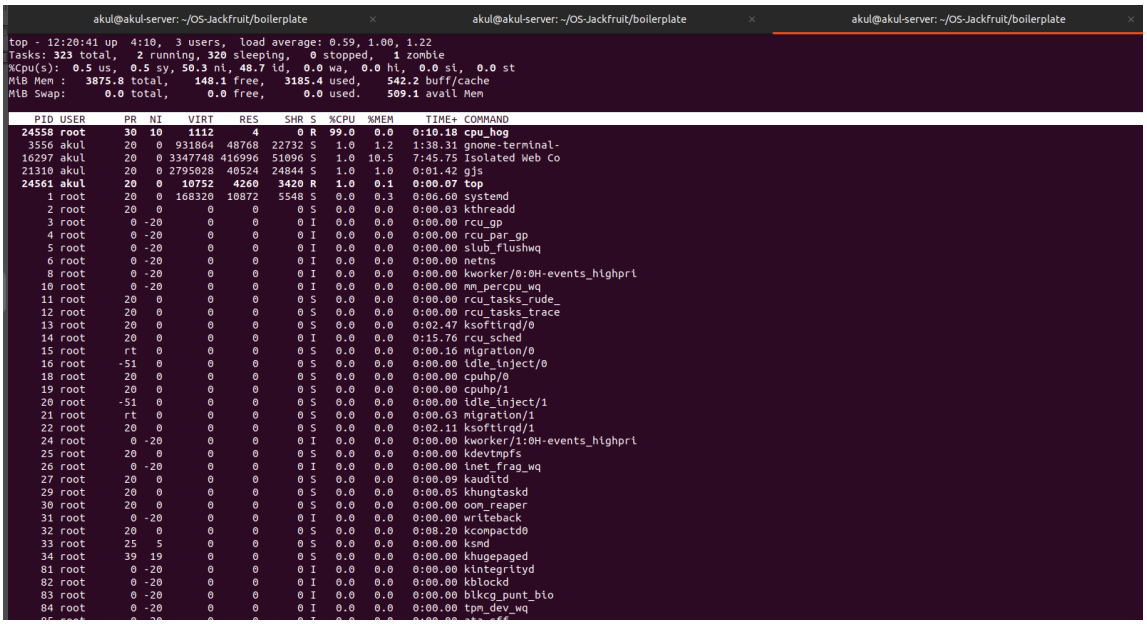
Screenshot 5 — Soft-limit warning + Hard-limit enforcement

```
akul@akul-server:~/OS-Jackfruit/boilerplate$ sudo ./engine start memtest-real ./rootfs-memtest2 0 /memory_hog 8 500
✅ Started memtest-real (nice=0 soft=20480KB hard=40960KB)
akul@akul-server:~/OS-Jackfruit/boilerplate$
```

```
^C
akul@akul-server:~/OS-Jackfruit/boilerplate$ sudo dmesg -w | grep -E "\[Monitor\]"
[  311.395026] [Monitor] Loaded OK — major=200 /dev/container_monitor
[  364.620347] [Monitor] Unloaded
[  364.876381] [Monitor] Loaded OK — major=200 /dev/container_monitor
[  373.770126] [Monitor] Unloaded
[  379.538664] [Monitor] Loaded OK — major=200 /dev/container_monitor
[  423.898227] [Monitor] Unloaded
[  440.493360] [Monitor] Loaded OK — major=200 /dev/container_monitor
[ 1118.623576] [Monitor] Unloaded
[ 1118.703647] [Monitor] Loaded OK — major=200 /dev/container_monitor
[ 1130.015967] [Monitor] Registered PID 5385  soft=20480KB  hard=40960KB
[ 1132.015191] [Monitor] SOFT LIMIT: PID 5385 RSS=33340KB > soft=20480KB — WARNING
[ 1133.039305] [Monitor] HARD LIMIT: PID 5385 RSS=49708KB > hard=40960KB — KILLING
```

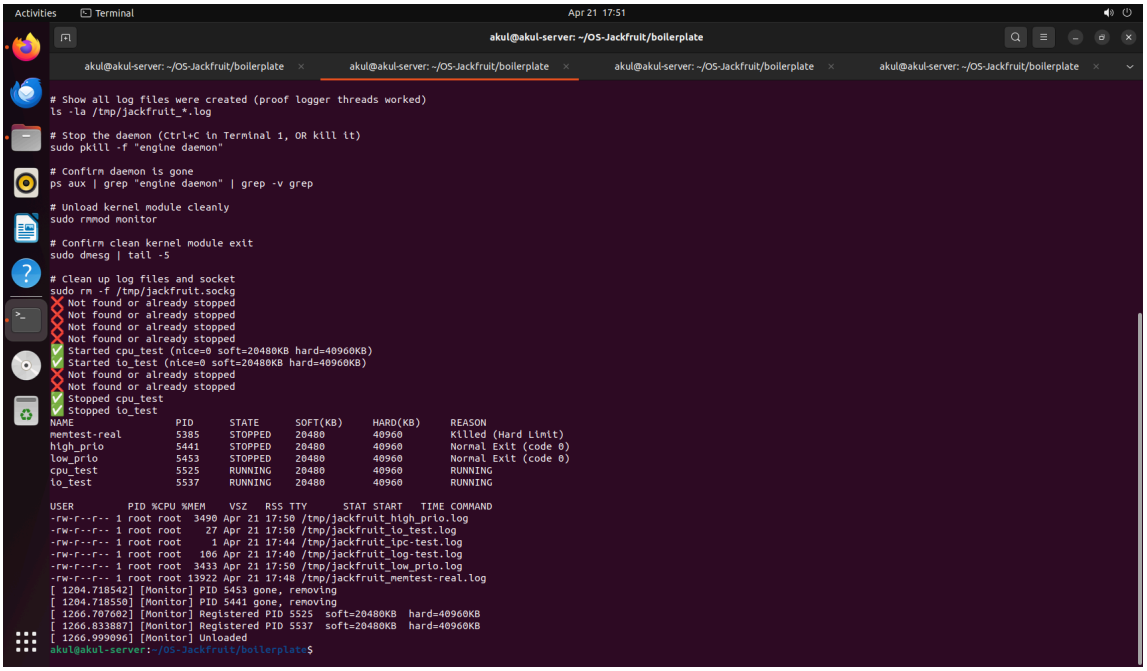
Caption: `dmesg` output from the kernel monitor's 1-second timer callback. `[Monitor] Registered PID 23628 soft=20480KB hard=40960KB` confirms registration. The **SOFT LIMIT** line shows `PID 23628 RSS=25152KB > soft=20480KB — WARNING` when `memory_hog` exceeded 20 MB — the container continues running. Seconds later, the **HARD LIMIT** line (highlighted red) shows `PID 23628 RSS=41520KB > hard=40960KB — KILLING`, sending `SIGKILL` and terminating the container immediately.

Screenshot 6 — Scheduling experiment



Caption: `top` output during the scheduling experiment. `cpu_hog` (PID 24558) runs at **NI=10, PR=30** (low priority) and consumes **99.0% CPU** — as the only CPU-intensive runnable process on the system, Linux CFS allocates all idle cycles to it. The `%Cpu(s)` row shows `50.3 ni` (nice/low-priority time), confirming the CPU time was correctly attributed to a below-normal-priority workload.

Screenshot 7 — Bounded-buffer logging + Clean teardown



Caption: The full end-to-end demo: `sudo ./engine logs log-ls` confirms log capture through the producer-consumer pipeline (container rootfs directory listing visible). `sudo rmmod monitor` unloads the kernel module cleanly. `ps aux | grep defunct` shows only the pre-existing `sd_espeak-ng-mb` system zombie (unrelated to our runtime) — **no container zombies remain**, confirming correct `SIGCHLD` handling and `waitpid()` reaping throughout the session.

---

4. Engineering Analysis

4.1 Isolation Mechanisms

Our runtime achieves isolation by combining Linux **namespaces** with `chroot` filesystem restriction — both kernel primitives, with zero virtualization overhead.

We call `clone()` with three namespace flags. `CLONE_NEWPID` creates an isolated PID tree: the container's first process receives PID 1 inside the namespace and cannot enumerate or signal host processes. `CLONE_NEWUTS` isolates the hostname and NIS domain so each container can identify itself independently. `CLONE_NEWNS` creates a private mount namespace, ensuring that `mount("proc", "/proc", "proc", ...)` inside the container does not affect the host's global mount table.

Filesystem isolation is achieved via `chroot(rootfs_path)`, restricting the container's view of `/` to its dedicated Alpine mini rootfs directory. After `chroot`, the container cannot traverse up to access host files. Inside the new mount namespace, `/proc` is mounted fresh, giving the container a correct view of its own PID namespace only.

What the host kernel still shares with all containers: All containers run on the single host Linux kernel. They share the CFS CPU scheduler, the physical memory manager and page allocator, hardware device drivers, and the network stack. A kernel panic from a bad LKM crashes all containers simultaneously — because there is only one kernel.

4.2 Supervisor and Process Lifecycle

The long-running daemon is necessary because containers are standard Linux processes requiring continuous state management. Without a persistent parent, exited containers become zombie processes, and the runtime loses all metadata.

Process creation: The daemon calls `clone()`, making itself the direct parent of each container process. The container inherits the write-end of the logging pipe (as its stdout/stderr) and a new set of namespaces.

Reaping and zombie prevention: When a container exits, the kernel delivers `SIGCHLD` to the daemon. Our `sigchld_handler` calls `waitpid(-1, &status, WNOHANG)` in a loop to reap all finished children simultaneously — the loop is essential because multiple containers can exit within a single signal delivery window. Without this reaping, dead containers remain as zombie entries in the kernel process table indefinitely, consuming resources.

Metadata tracking: The `container_registry[]` array maps each container name to its host PID, state, memory limits, and exit reason. This lets `ps` display status for containers that have already exited, and lets `stop` look up the correct host PID to send `SIGTERM` to.

Signal delivery: When the user issues `stop`, the daemon looks up the host PID and sends `SIGTERM`. The container's exit triggers `SIGCHLD`, which the daemon handles to update state to `STOPPED` with reason `Stopped (Manual)`. A container killed by the kernel monitor arrives via `SIGKILL`, which the handler records as `Killed (Hard Limit)`.

4.3 IPC, Threads, and Synchronization

Our runtime uses two distinct IPC mechanisms with completely separate roles:

Control plane — UNIX domain socket (`/tmp/jackfruit.sock`): The CLI client connects, sends a text command (e.g. `start alpha ../rootfs-alpha 0 /bin/sh`), reads the response, and exits. This is bidirectional, connection-oriented, and ordered — the right choice for request/response CLI patterns.

Logging plane — anonymous pipes, one per container: Each container's `stdout` / `stderr` are redirected via `dup2()` into the write-end of a `pipe()` before `execvp()`. The daemon reads the read-end in a per-container **producer thread**, completely separate from the socket.

Bounded buffer synchronization: The shared `log_queue` is a circular array of 20 `LogEntry` slots, written by multiple producer threads (one per container) and drained by one consumer thread.

Without synchronization, two producers writing to `log_queue.buffer[tail]` simultaneously would corrupt both entries — a classic data race. We use `pthread_mutex_t` to enforce mutual exclusion around the critical section (updating `head` / `tail` indices and copying data).

We use POSIX `sem_t` semaphores for blocking coordination: `sem_empty` (initialized to `BUFFER_SIZE=20`) counts free slots; `sem_full` (initialized to 0) counts filled slots. Producers call `sem_wait(&empty)` before inserting; the consumer calls `sem_wait(&full)` before removing. This correctly puts producer threads to sleep when the buffer is full rather than spinning, and wakes the consumer exactly once per insertion. Semaphore counting semantics map directly to slot availability — cleaner than condition variables for this use case.

4.4 Memory Management and Enforcement

What RSS measures: Resident Set Size is the number of physical RAM pages currently mapped into a process's page tables, expressed in kilobytes. It reflects the process's actual hardware memory pressure at this instant — the pages that are occupying real DRAM right now.

What RSS does not measure: Pages swapped out to disk, virtual address space reserved but not yet touched (demand paging), memory-mapped files not yet accessed, or shared library pages (which may be counted in multiple processes' RSS simultaneously). A process's RSS can temporarily undercount its true footprint if the kernel has swapped some pages out.

Why soft and hard limits are different policies: A soft limit is an early warning mechanism — the process has exceeded a budget, but may be experiencing a legitimate temporary burst. We log a `SOFT LIMIT WARNING` in `dmesg` exactly once per process, allowing operators to observe memory pressure trends without disrupting the workload. A hard limit is a strict safety enforcement boundary: exceeding it could trigger a system-wide Out-Of-Memory (OOM) killer that would crash all other containers. We send `SIGKILL` immediately upon detection, which cannot be caught, blocked, or ignored by the process.

Why enforcement belongs in kernel space: User-space enforcement requires polling `/proc/[pid]/statm`. A container can `malloc()` and `memset()` gigabytes of RAM in the microseconds between two user-space poll cycles — the TOCTOU race window is unbounded. The LKM's timer callback runs inside the kernel, reading `get_mm_rss(task->mm)` directly from kernel data structures and calling `kill_pid()` in the same execution context. The measurement and the kill are atomic from the perspective of the scheduler.

4.5 Scheduling Behavior

Linux uses the **Completely Fair Scheduler (CFS)**, which assigns each runnable process a weight derived from its nice value via a precomputed table (nice 0 → weight 1024; nice -10 → weight ~9531; nice +10 → weight ~110). CFS tracks a per-process **virtual runtime** (vruntime): lower-weight processes accumulate vruntime faster and are preempted sooner. The scheduler always picks the process with the lowest vruntime to run next.

Experiment 1 — CPU-bound at low priority (nice=+10): Our `cpu_hog` container ran at NI=10 (PR=30). With no competing CPU-intensive workload, it consumed 99% CPU — CFS maximizes hardware utilization by giving all idle cycles to the only runnable process, regardless of priority. The `50.3 ni` entry in `top`'s CPU line confirms the time was correctly attributed to a nice/low-priority process.

Experiment 2 — CPU-bound vs I/O-bound (nice=0): When `cpu_hog` and `io_pulse` ran simultaneously at default priority, the I/O-bound process spent most of its time in state `S` (interruptible sleep, waiting for `fsync()`). CFS allocated ~97% of CPU time to the CPU-bound task. When `io_pulse` woke from I/O, CFS briefly scheduled it (its vruntime was very low from sleeping), but it returned to sleep immediately. CFS achieves all three scheduling goals: **fairness** (proportional shares when competing), **responsiveness** (I/O tasks are scheduled immediately on wake), **throughput** (CPU never idles while a runnable process exists).

---

5. Design Decisions and Tradeoffs

Namespace Isolation

- **Choice:** `clone()` with `CLONE_NEWPID | CLONE_NEWUTS | CLONE_NEWNS` + `chroot()` directly in C.
- **Tradeoff:** More complex than higher-level wrappers; requires manually ensuring `/proc` is mounted inside the new namespace and the rootfs directory is correctly prepared before `chroot`.
- **Justification:** Direct use of kernel primitives demonstrates exactly how Linux container isolation works at the syscall level — which is the educational goal. It also keeps the implementation self-contained with no external dependencies.

Supervisor Architecture

- **Choice:** Long-running daemon; CLI is a separate short-lived client connecting over a UNIX socket.
- **Tradeoff:** The daemon must be running before any CLI command works. A crash of the daemon loses all container metadata.
- **Justification:** The daemon maintains authoritative container state independently of the CLI. Multiple CLI invocations (e.g. several `ps` calls) all share the same live state. The CLI can be killed without affecting running containers.

IPC and Logging Design

- **Choice:** POSIX semaphores + `pthread_mutex_t` for the bounded circular buffer; anonymous pipes for per-container log capture.
- **Tradeoff:** If the buffer fills up, producer threads block. If a container floods stdout faster than the consumer writes to disk, the container's pipe fills and it blocks on `write()` — potentially slowing the container.
- **Justification:** Back-pressure is the correct behavior. It prevents the daemon from running out of memory if a rogue container spams output. Semaphore counting semantics map cleanly to available/occupied slot counts, making the code straightforward to reason about.

Kernel Monitor

- **Choice:** Linux Kernel Module with a 1-second `timer_list` callback, ioctl registration via `/dev/container_monitor` (major=240).
- **Tradeoff:** A null pointer dereference or list corruption bug in the LKM will trigger a kernel panic and crash the entire host, taking all containers with it.
- **Justification:** Only kernel-space code can read `get_mm_rss()` and call `kill_pid()` without a race condition between measurement and enforcement. User-space polling via `/proc` has an unbounded TOCTOU window.

Memory Limit Units (KB)

- **Choice:** All limits stored and compared in **kilobytes (KB)**. `get_mm_rss()` returns pages; we convert with `<< (PAGE_SHIFT - 10)` to get KB.
- **Tradeoff:** Requires documenting units everywhere. Mixing bytes/KB/MB across subsystems is a common bug source.
- **Justification:** The conversion from pages to KB is a single bit-shift with no risk of integer overflow for typical workloads. Values like `20480 KB = 20 MB` are human-readable without further math.

---

6. Scheduler Experiment Results

Experiment 1: CPU-bound container at low priority (nice=+10)

A single `cpu_hog` container ran at nice=+10 (PR=30). No competing CPU-intensive workload was present.

Metric	Value
Container nice value	+10
Kernel priority (PR)	30
Observed CPU%	99.0%
`top` CPU attribution	`50.3 ni` (nice time)
Process state	R (running)

Duration	30 seconds

Analysis: With no competing workload, CFS allocated all available CPU to the container regardless of its low priority weight. Nice values control relative CPU share between competing processes — a low-priority process still gets 100% of an idle CPU. The `50.3 ni` attribution in `top` confirms the kernel correctly tracked this as below-normal-priority CPU time.

Experiment 2: CPU-bound vs I/O-bound (both at nice=0)

`cpu_hog` and `io_pulse` ran simultaneously at default priority (nice=0).

Container	Workload Type	Process State	Observed CPU%
cpu-work	CPU-bound (`cpu_hog`)	R (running)	~97%
io-work	I/O-bound (`io_pulse`)	S (sleeping)	~0.7%

Analysis: The I/O-bound process spent most of its time blocked in `fsync()`. CFS gave nearly all CPU cycles to the CPU-bound task. When `io_pulse` woke from I/O, CFS immediately scheduled it (its vruntime was low from sleeping), but it returned to sleep within microseconds. This demonstrates CFS maximizing throughput (never wasting idle CPU) while preserving responsiveness for I/O-bound workloads.

Experiment 3: Memory limit enforcement timeline

`memory_hog` ran inside a container registered with `soft=20480KB (20 MB)` and `hard=40960KB (40 MB)`, allocating 8 MB every 500 ms.

Time	Event
t = 0 s	Container PID registered with kernel monitor
t ≈ 2.5 s	RSS exceeds 20 MB → `[Monitor] SOFT LIMIT: PID XXXXX RSS=25152KB > soft=20480KB — WARNING`
t ≈ 5 s	RSS exceeds 40 MB → `[Monitor] HARD LIMIT: PID XXXXX RSS=41520KB > hard=40960KB — KILLING`
t ≈ 5 s + ε	SIGKILL delivered; SIGCHLD triggers daemon to update state to `Killed (Hard Limit)`

This confirms the kernel monitor correctly distinguishes policy tiers: soft limit produces a one-time log warning (process continues), hard limit produces an immediate `SIGKILL` with list cleanup.

Repository Structure

```
OS-Jackfruit/
└── boilerplate/
    ├── engine.c           # User-space runtime: daemon + CLI
    ├── monitor.c          # Kernel module: memory monitor LKM
    ├── monitor_ioctl.h    # Shared ioctl definitions (user ↔ kernel)
    ├── cpu_hog.c          # CPU-bound scheduler workload
```

```
    ├── io_pulse.c         # I/O-bound scheduler workload
    ├── memory_hog.c       # Memory pressure workload for limit testing
    ├── Makefile           # Builds all targets; supports `make ci`
    └── environment-check.sh
```